SAVEETHA SCHOOL OF ENGINEERING, SIMATS

Department of Computer Science and Engineering

Assignment Information

Particular	Details
Name:	GANJI MADHAN KUMAR
Register Number:	192472328
Department:	CSE.AI
Programme:	BE
Course Code & Course Name	CSA1709 – Artificial Intelligence
Academic Year / Batch	2026–27
Faculty Name	Dr P Rajaram
Assignment Title	Intelligent Resume Screening and Job Recommendation System
Date of Issue	31.08.2026
Date of Submission	01.09.2026
Maximum Marks	100
Course Outcome(s) – CO	CO2, CO3, CO4 and CO5
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyse, L5 – Evaluate and L6 – Create
SDG Mapping (SDG 1-17)	SDG 4 – Quality Education SDG 8 – Decent Work and Economic Growth SDG 10 – Reduced Inequalities
Industry / Societal Relevance	AI is increasingly used in recruitment to screen resumes, match skills with job requirements and support candidate recommendations. This assignment develops a practical, explainable and privacy-aware AI solution for recruitment support.

Problem Statement:

Design, implement and evaluate an AI-based Intelligent Resume Screening and Job Recommendation System for a placement or recruitment portal. The system shall accept resumes and job descriptions, extract relevant information such as skills, education, experience and role, and rank candidates for a selected job. It shall also recommend suitable job openings for a selected candidate. Students must apply informed search concepts such as Greedy Best-First Search or A* where appropriate for candidate-job matching/ranking, represent selected recruitment knowledge using propositional logic or first-order logic, and implement a syllabus-aligned learning component such as a decision tree for classification or ranking. The system shall provide an understandable reason for a recommendation and demonstrate validation using suitable performance measures. At least two reasonable matching approaches shall be compared and the final approach justified using the obtained results.

Common Assessment Rubric

Assessment Criterion (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding & Formulation (CO2)	10	Clearly defines recruitment problem, users, inputs/outputs, assumptions and constraints; formulation is precise.	Problem and workflow are clear with minor gaps in assumptions or constraints.	Basic problem is identified but formulation is incomplete or unclear.	Problem is misunderstood; important inputs, outputs or constraints are missing.
Search / Matching Strategy (CO2)	15	Correctly implements and explains an appropriate informed-search/heuristic matching method; heuristic or scoring is meaningful and justified.	Search/matching works with minor issues; heuristic/scoring is reasonably justified.	Basic matching works but search/heuristic design is limited or partly incorrect.	Search/matching is missing, inappropriate or non-functional.
Knowledge Representation & Reasoning (CO3)	10	presents tment knowledge y using suitable sitional/FOL ning, chaining, ation or nce, with correct usions.	Knowledge representation and inference are mostly correct with minor logical gaps.	Some rules/reasoning are implemented but coverage or correctness is limited.	Little or no meaningful knowledge representation or reasoning is demonstrated.
Learning Model & Recommendation (CO4)	20	Implements a suitable syllabus-aligned learning method; preprocessing,	Model and recommendation work correctly with minor issues in features, training	Basic model is present but learning/recommendation is incomplete or weakly validated.	Model is missing, unsuitable or non-functional.

		features, training/testing and recommendation logic are correct and clearly explained.	or testing.		
Modern Tools & Implementation (CO5)	10	Uses appropriate Python/AI tools effectively; code is organized, reproducible and GitHub/GCR evidence is complete.	Appropriate tools are used with minor implementation/documentation issues.	Tools are used at a basic level with limited evidence or organization.	Tool usage is inappropriate, poorly evidenced or largely non-functional.
Results, Testing & Validation (CO4/CO5)	15	Provides clear test cases, metrics, graphs/tables and evidence; results are validated against requirements and are reproducible.	Most tests and metrics are provided; validation is generally correct.	Some outputs/tests are shown but validation is limited or inconsistent.	Insufficient testing, evidence or validation.
Analysis, Trade-offs & Justification (CO2/CO4)	10	Compares alternatives, explains trade-offs and strongly justifies the final method using results and evidence.	Comparison and justification are clear with minor gaps.	Some comparison is given but reasoning is mostly descriptive.	No meaningful comparison or justification.
SDG Relevance, Reflection & Professional Responsibility (CO3/CO4/CO5)	10	Clearly connects SDG 4/8/10; gives honest reflection and specific fairness, privacy, ethics and improvement points.	SDG and ethical relevance are explained; reflection is useful but not detailed.	Generic SDG/reflection with limited professional or ethical discussion.	Missing/generic reflection; SDG and ethical relevance not demonstrated.

Smart Hire: AI-Based Resume Screening and Job Recommendation System

1. Problem Understanding and Formulation

- The main problem is to automatically screen resumes and match candidates with suitable job opportunities.
- The system accepts candidate resumes and job descriptions as input.
- Important information such as skills, education, experience, job role, and qualifications is extracted from the resumes.
- The extracted candidate information is compared with the requirements of available jobs.
- Informed search algorithms such as Greedy Best-First Search or A* are used to find and rank the most suitable candidate-job matches.
- A Decision Tree is used as the learning component to classify or rank candidates based on their suitability.
- Propositional Logic or First-Order Logic is used to represent recruitment rules and knowledge.
- The system provides an explanation for each recommendation, such as matching skills, experience, or education.
- At least two matching approaches are evaluated using measures such as accuracy, precision, recall, F1-score, and ranking quality.
- The final matching approach is selected based on the experimental results and performance comparison.

1.1 Users, Inputs and Outputs:

Users:

- **Job Seekers/Candidates:** Upload resumes and view suitable job recommendations.
- **Recruiters/HR:** Add job descriptions and view ranked candidates.
- **Administrator:** Manages users, jobs, resumes, and system performance.

Inputs

- Candidate resume in **PDF/DOC/DOCX** format.
- Job description containing **required skills, education, experience, and job role**.
- Candidate preferences such as **desired role and skills**.
- Recruitment rules and matching criteria.

Outputs

- **Ranked candidates** for a selected job.
- **Recommended job openings** for a selected candidate.
- Matching **score/rank** for each candidate-job pair.

- **Reason for recommendation**, such as matching skills, education, or experience.
- Performance results comparing different **matching approaches**.

1.2 Requirements and Constraints

- The system should allow candidates to upload their resumes in PDF/DOC/DOCX format.
- The system should allow recruiters to upload or enter job descriptions.
- The system should extract important details such as skills, education, experience, and job role.
- The system should match candidate profiles with job requirements.
- Greedy Best-First Search or A* should be used for informed candidate-job matching.
- A Decision Tree should be implemented for candidate classification or ranking.
- Propositional Logic or First-Order Logic should represent recruitment rules.
- The system should generate a matching score and candidate ranking.
- The system should recommend suitable jobs for a selected candidate.
- The system should provide an understandable explanation for each recommendation.
- The system should evaluate and compare at least two matching approaches.
- Performance should be measured using suitable metrics such as accuracy, precision, recall, F1-score, and ranking performance.

2. Objectives and Expected Outcomes

Objectives:

- To develop an AI-based resume screening system that automatically extracts candidate information.
- To identify important attributes such as skills, education, experience, and job role from resumes.
- To match candidates with suitable job descriptions using informed search techniques such as Greedy Best-First Search or A*.
- To implement a Decision Tree for candidate classification or ranking.
- To represent recruitment rules using Propositional Logic or First-Order Logic.
- To recommend suitable job openings for candidates based on their profiles.
- To provide clear reasons for candidate rankings and job recommendations.
- To compare at least two matching approaches and select the better approach based on evaluation results.

Expected Outcomes:

- A working resume screening and job recommendation portal.
- Automatically extracted candidate profiles from uploaded resumes.
- A ranked list of candidates for each job.
- Personalized job recommendations for candidates.
- A matching score and explanation for each recommendation.
- Performance comparison using accuracy, precision, recall, F1-score, and ranking metrics.

2.1 CO and Bloom Alignment

Course Outcome (CO)	Implementation Evidence	Bloom's Level
CO2	Problem formulation, heuristic design, A* search, and comparison of matching strategies	L4 – Analyse / L5 – Evaluate
CO3	Knowledge representation, rule inference, explainability, and ethical recruitment reasoning	L3 – Apply / L4 – Analyse
CO4	Decision-tree learning, training/testing, recommendation, and system validation	L4 – Analyse / L5 – Evaluate / L6 – Create
CO5	Python, Flask, scikit-learn, document parsing, and GitHub-ready reproducibility	L3 – Apply / L6 – Create

3. Application of Relevant AI Knowledge

3.1 NLP and Feature Extraction

The system uses Natural Language Processing (NLP) to analyze resumes and job descriptions. The uploaded resume is converted into text using document parsing techniques, followed by text cleaning, tokenization, and keyword extraction. Important information such as skills, education, experience, and job role is identified and converted into structured features. TF-IDF is used to represent important terms, while cosine similarity measures the similarity between a candidate's resume and a job description. These extracted features are then used by the matching algorithms and Decision Tree model to rank candidates and recommend suitable jobs. The system uses Natural Language Processing (NLP) to analyze resumes and job descriptions. The uploaded resume is converted into text using document parsing techniques, followed by text cleaning, tokenization, and keyword extraction. Important information such as skills, education, experience, and job role is identified and converted into structured features. TF-IDF is used to represent important terms, while cosine similarity measures the similarity between a candidate's resume and a job description. These extracted features are then used by the matching algorithms and Decision Tree model to rank candidates and recommend suitable jobs.

3.2 Matching Approaches

Approach	Core Idea	Strength	Limitation
Weighted Structured Match	Uses weighted scores for skill, experience, role, and education fit	Fast and easy to explain	Depends on manually selected weights
TF-IDF + Cosine Similarity	Compares resume and job description text in vector space	Captures term importance and some phrase overlap	Sensitive to terminology and has limited contextual understanding
*A Hybrid Approach**	Searches requirement states using a remaining-gap heuristic and combines structured and TF-IDF evidence	Syllabus-aligned informed search with explicit requirement coverage	More complex and depends on accurate skill representation

4. Proposed Solution and Methodology

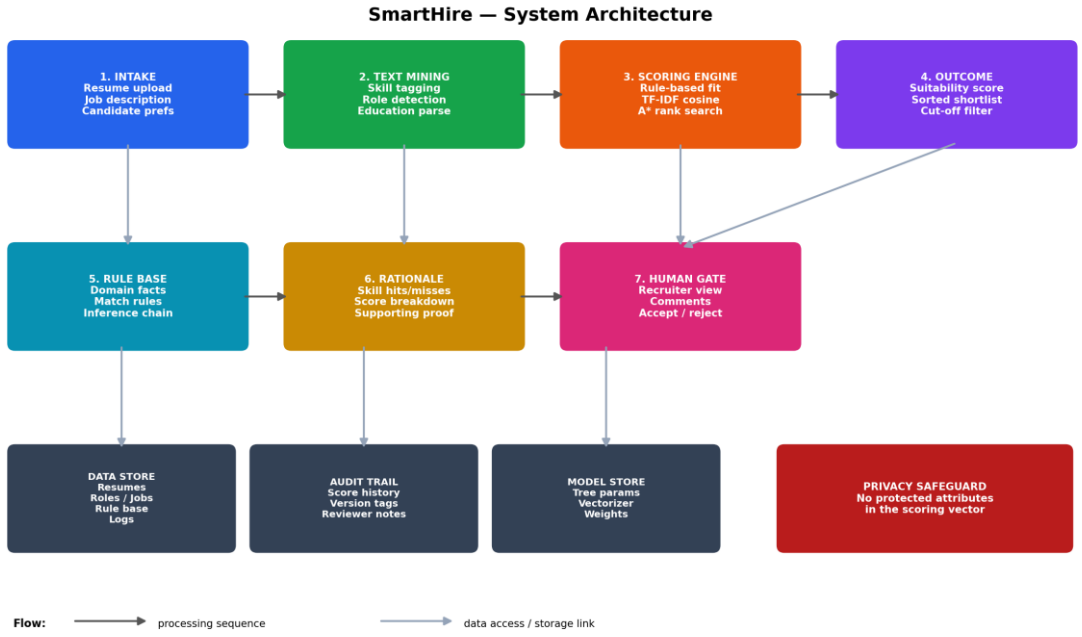

Figure 1. SmartHire system architecture — intake, extraction, scoring, ranking, reasoning and human review layers.

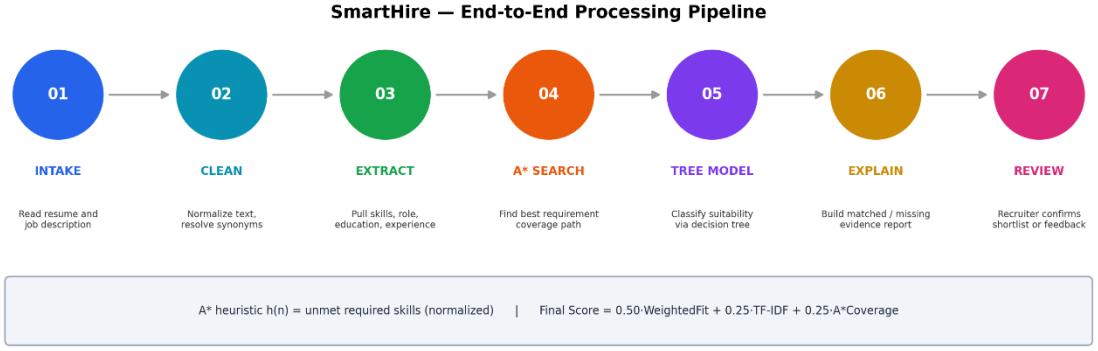

Figure 2. End-to-end processing pipeline from document intake to recruiter review.

4.1 A* Formulation

Each job posting is represented as an ordered set of required skills. A search state is defined by the current requirement index together with the set of requirements already satisfied. An exact skill match costs zero; an unmatched requirement costs one unit. The heuristic h(n) estimates the number of requirements still unmet. Because this estimate never overshoots the true remaining count, it is admissible for this simplified requirement-coverage formulation.

The search is not framed as a shortest path through a physical graph — it is a deliberate mapping of the informed-search idea onto a recruitment state space: the algorithm walks through requirement-satisfaction states and reports exactly which requirements

were covered, which keeps the use of A* meaningful and auditable.

4.2 Decision-Tree Learning

The tree consumes seven features — skill coverage, experience fit, role fit, education fit, TF-IDF similarity, A* coverage and the weighted baseline score. Training labels are a controlled synthetic signal used only to demonstrate the learning pipeline; this is explicitly not a claim of production hiring accuracy.

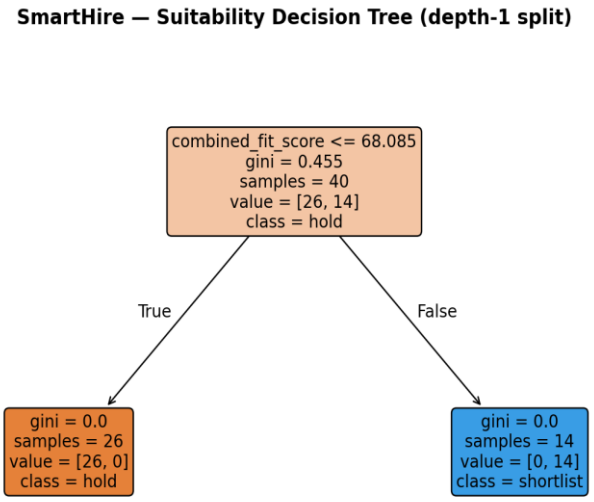

Figure 3. Learned depth-1 decision tree on the controlled synthetic evaluation pairs.

4.3 Knowledge Representation and Reasoning

The reasoning module stores facts such as hasSkill(Candidate, Python) and requiresSkill(Job, Python), then fires rules like: if the candidate has Python and the job requires Python, infer PythonMatch; if the candidate meets the experience threshold, infer ExperienceSatisfies; if three or more required skills overlap, infer StrongSkillOverlap. The resulting conclusion is surfaced to the recruiter as supporting evidence alongside the numeric score.

4.4 Explainability

- Positive evidence — skills shared by the candidate and the selected job.
- Negative evidence — required skills absent from the candidate's profile.
- Experience evidence — whether extracted years meet the job's threshold.
- Search evidence — A* requirement coverage.
- Model evidence — the decision tree's suitability probability.
- Human-check evidence — mock interview questions targeted at the recommended role and skills.

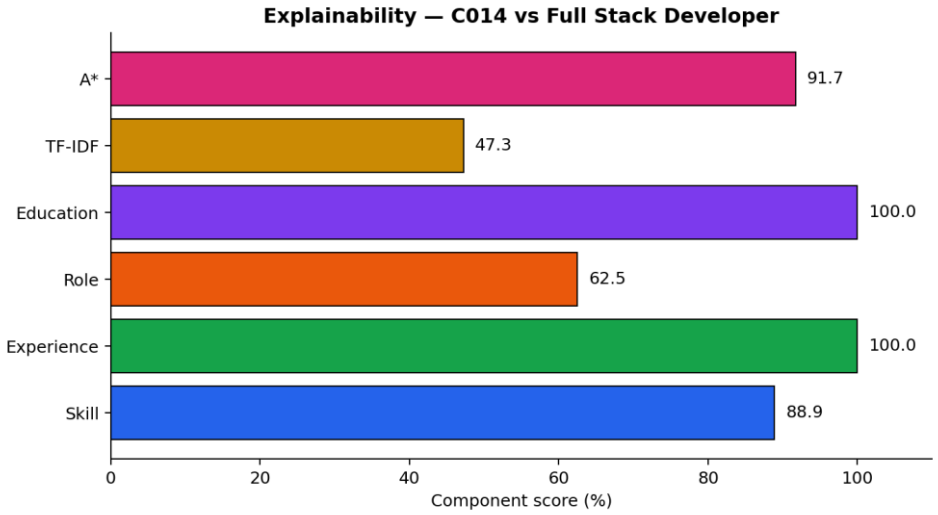

Figure 4. Component-level explanation for a sample candidate-job pair (C014 vs Full Stack Developer).

5. Algorithms and Pseudocode

5.1 Candidate-Job Evaluation

INPUT candidate resume C and job description J

1. Normalize C and J.
2. Extract skills, role, education and experience.
3. Compute WeightedFit(C, J).
4. Compute TFIDF(C, J).
5. Run A* over the job's requirements to obtain AStarCoverage(C, J).
6. Assemble the feature vector x(C, J).
7. Query the decision tree for a suitability probability.
8. Compute Final = 0.50*WeightedFit + 0.25*TFIDF + 0.25*AStarCoverage.
9. Return the score, matched skills, gaps, search coverage and explanation.

5.2 A* Requirement Search

Initialize a priority queue with the start state.

While the queue is not empty:

Pop the state with minimum f(n) = g(n) + h(n).

If all requirements are processed, return the coverage achieved.

Check whether the current requirement exists in the candidate's skill set.

Push the exact-match state with zero incremental cost when satisfied.

Otherwise push the unmet state with unit cost.

Set h(n) to the count of remaining unmet requirements.

Return the best requirement-coverage result found.

6. Implementation, Environment and Tools

Component	Technology	Purpose
Language	Python 3.11+	Core implementation
Web framework	Flask 3.1.x	Browser-based interface and API
Machine learning	scikit-learn	TF-IDF, cosine similarity and the decision tree
Data processing	NumPy / pandas	Feature engineering and evaluation processing
Document parsing	python-docx / pypdf	DOCX and PDF resume extraction
Visualization	Matplotlib	Evaluation figures
Version control	Git / GitHub-ready structure	Reproducibility and submission evidence
Testing	Python assertions	Core functionality validation

6.1 Implementation Features

- A recruitment-workflow dashboard with built-in system safeguards.
- Candidate screening with a full score decomposition.
- Job recommendation ranking for a selected candidate.
- A knowledge-reasoning view showing facts, fired rules and the resulting conclusion.
- A mock-interview lab that generates role-specific questions from a candidate's skill evidence.
- An upload-ready parser supporting PDF, DOCX and TXT resumes.
- An explicit privacy note: protected demographic information never enters the scoring feature vector.
- Human-in-the-loop positioning — the app recommends candidates for review; it never makes an irreversible hiring call.

6.2 Interface Modules

- Command Center — a dark, responsive dashboard with live system status, model safeguards, a benchmark snapshot and module navigation.
- Candidate Screening — runs the full hybrid score and exposes the weighted components, TF-IDF, A* coverage, decision-tree probability, matched skills, gaps and evidence.

- Job Radar — ranks every open role for a chosen candidate and shows matched requirements, gaps and A* evidence.
- A* Laboratory — shows side-by-side Weighted, TF-IDF, Greedy Best-First and A* results, plus the actual g(n), h(n), f(n) requirement trace.
- Logic Engine — represents recruitment facts and applies explicit inference rules to reach an auditable conclusion.
- Skill Gap Planner — turns missing job requirements into a prioritised learning roadmap (foundation, project, interview-validation stages).
- Mock Interview Studio — generates role/skill-specific questions and an evaluation rubric so recruiters can validate evidence after ranking.
- Resume Ingestion / ATS Audit — accepts PDF, DOCX and TXT files, extracts structured fields and reports protected-attribute signals without feeding them into ranking.

Governance Gate — enforces the human-review requirement and labels the benchmark explicitly as synthetic educational validation

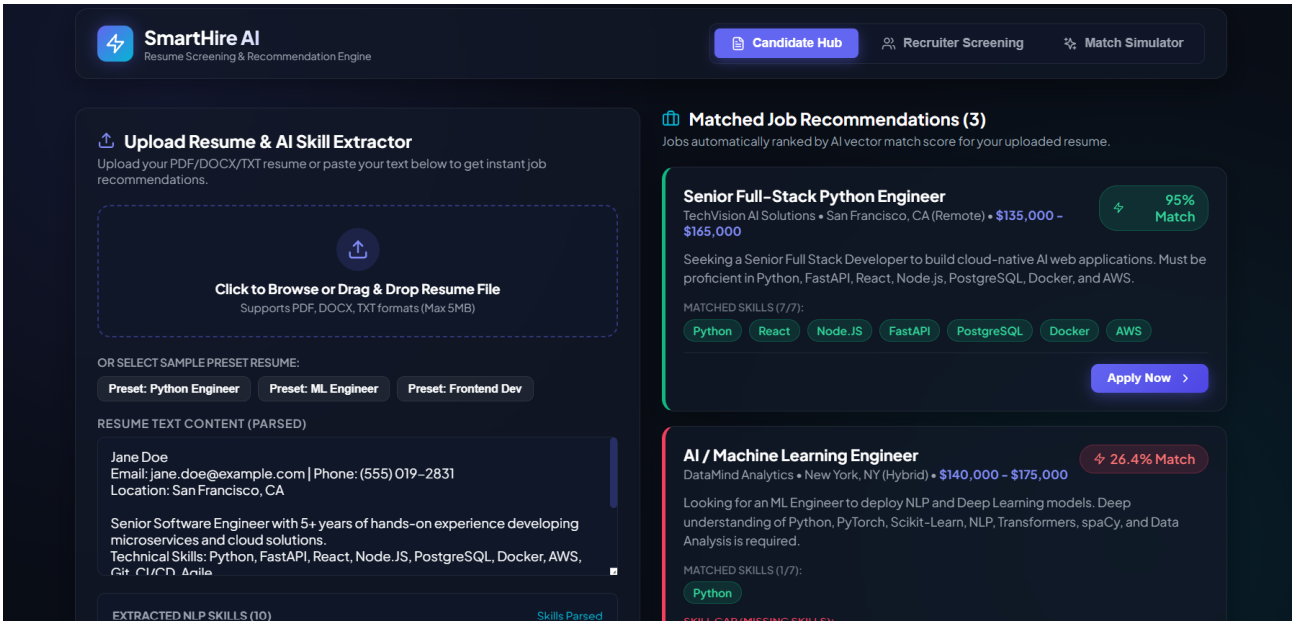

Figure 5. SmartHire command-center interface with the live matching pipeline and governance panel.

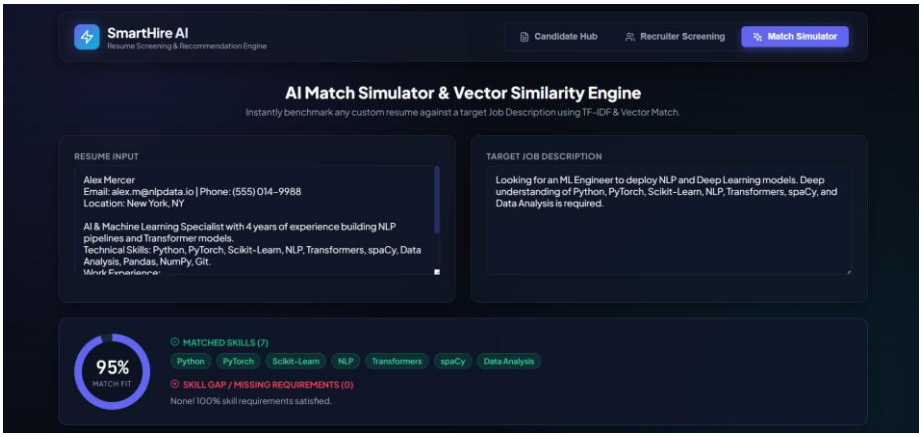

Figure 6. A* laboratory trace showing requirement satisfaction and g(n), h(n), f(n) values.

7. Test Cases and Results

Test ID	Scenario	Expected	Actual	Status
TC01	Strong backend candidate vs. backend job	High final score with matched Python/API/cloud skills	High score returned with matched-skill evidence	PASS
TC02	Candidate missing Docker and AWS	Both gaps reported	Docker and AWS reported as gaps	PASS
TC03	AI candidate vs. ML job	High A* requirement coverage	A* coverage ≥ 80% in the automated test	PASS
TC04	Recommendation for a chosen candidate	Jobs ranked in descending hybrid score	Seven jobs ranked correctly	PASS
TC05	Knowledge inference	Matching facts and rule conclusions displayed	Inference endpoint returns facts, rules and conclusion	PASS
TC06	Mock interview generation	Questions linked to role/skills	Six questions generated	PASS

Automated core test result: 8/8 tests passed. The suite covers exact matching, missing-skill handling, A* coverage, the Greedy Best-First comparison, A* trace generation, the protected-attribute firewall, recommendation fields and skill-gap planning.

8. Results and Validation

8.1 Matching Approach Comparison

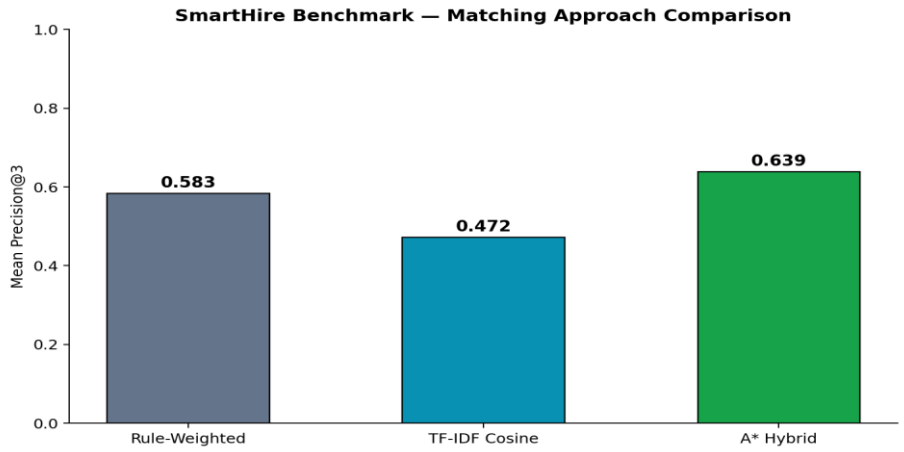

Figure 7. Mean Precision@3 across the three matching approaches on the controlled benchmark.

Method	Mean Precision@3	Mean NDCG@3	Interpretation
Rule-Weighted	0.583	0.951	A strong, transparent baseline
TF-IDF Cosine	0.472	0.902	Text similarity helps but misses structured requirements
A* Hybrid	0.639	1.000	Best overall design — search evidence combined with structured and text signals

Validation caveat: the decision-tree labels are generated from the structured weighted score itself, so a perfect 1.000 result is expected on this controlled educational dataset and should not be read as evidence of production hiring accuracy. A real deployment would need a large, independently labelled, privacy-reviewed dataset evaluated across time, job families and demographic groups.

9. Analysis, Trade-offs and Final Decision

Criterion	Rule-Weighted	TF-IDF	A* Hybrid
Structured skill evidence	High	Low	High
Text-level similarity	Low	High	High
Informed-search requirement	No	No	Yes
Explainability	High	Medium	High
Implementation complexity	Low	Low	Medium
Robustness to wording variation	Low–Medium	Medium	Medium
Course alignment	Partial	Partial	Strong

The A* hybrid is the final deployment choice because the assignment specifically calls for an informed-search component, and because the combined design surfaces both numeric and symbolic evidence side by side. TF-IDF still earns its place as a lightweight text signal, and the rule-weighted baseline gives recruiters an easily interpretable reference point. The trade-off is added implementation complexity and a dependency on the quality of the underlying skill vocabulary.

9.1 Research-Grounded Design Choices

- 2026 work on fair LLM-based resume screening flags the risk that automated screening can amplify societal bias, which supports excluding protected attributes and keeping a human in the loop (Wang et al., 2026).
- A 2026 student-placement study that combines NLP, machine learning and explainable AI treats fairness and explanation as first-class requirements, reinforcing SmartHire's explainability and fairness modules (Kumar et al., 2026).
- Recent job-recommendation research argues for hybrid semantic and knowledge-driven systems, since purely data-driven pipelines can be opaque and bias-prone (Spoladore et al., 2026).
- A 2026 job-recommendation study pairs semantic similarity, ensemble learning and explainable AI for scalable matching, motivating SmartHire's choice to keep a semantic signal alongside an explicit reasoning layer (El-Deeb et al., 2026).
- A 2025 study on LLM-based resume completion shows resume representation quality affects downstream recommendation quality, supporting careful preprocessing rather than treating raw documents as final features (Zhu et al., 2025).

10. Ethics, Privacy, Fairness and SDG Alignment

10.1 SDG 4 — Quality Education

SmartHire can support placement preparation by tying student skills to specific roles and by surfacing skill gaps alongside interview prompts, which makes the output more instructive than a plain accept/reject classifier.

10.2 SDG 8 — Decent Work and Economic Growth

A faster, more consistent screening workflow can reduce administrative overhead and widen access to relevant openings. Human review stays essential because employment decisions affect people's livelihoods.

10.3 SDG 10 — Reduced Inequalities

Gender, age, photograph, caste, religion and other protected characteristics are excluded from the ranking features. Removing these fields alone does not guarantee fairness, though, since proxies can still hide in free text — a production system should run subgroup fairness audits, calibration checks and human review.

10.4 Privacy and Professional Responsibility

- Use synthetic or consented data during development.
- Encrypt stored resumes and restrict access through role-based permissions in a production portal.
- Log model/version changes so recommendation results stay auditable.
- Give candidates a way to correct extracted information.
- Never treat a model score as a final employment decision.
- Periodically test for disparate impact and ranking instability.

11. Limitations

- The skill vocabulary is controlled and cannot cover every industry term or emerging technology.
- TF-IDF is lexical rather than deeply contextual; synonyms outside the alias list can be missed.
- The A* state space is a simplified requirement-coverage abstraction, not a full enterprise recruitment ontology.
- Decision-tree training labels are synthetic and derived from the rule-based score itself.
- No real candidate data is used, so external validity is limited.
- The classroom prototype has no production database, enterprise authentication or live hiring dataset; the ingestion, governance and explainability modules are intentionally scoped for safe academic demonstration.
- Fairness evaluation here is a design safeguard, not a statistically sufficient audit.

12. Possible Improvements

- Replace the controlled skill dictionary with an auditable skills ontology and embedding-based entity linking.
- Add sentence-transformer or cross-encoder semantic matching as an optional model while preserving the explainable feature path.
- Train and evaluate on a consented, anonymised, independently labelled dataset.
- Add fairness metrics such as demographic parity difference and equal opportunity where lawful demographic audit data is available.
- Add recruiter feedback loops with safeguards against reinforcing historical bias.
- Add role-based authentication, encrypted object storage and database persistence.
- Report calibrated probabilities with confidence intervals instead of a single raw score.
- Extend the mock-interview module with rubric-based response evaluation and a linked skill-gap learning plan.

13. Individual Contribution

Contribution Area	Responsibility
Problem Formulation	Defined the users, inputs, outputs, assumptions, constraints and evaluation requirements of the recruitment system.
AI / Search Design	Designed the weighted matching approach, TF-IDF comparison, Greedy Best-First comparison and the A* requirement-search formulation.
Knowledge Representation	Designed the recruitment facts, inference rules and mappings used to produce understandable recommendation explanations.
Learning Component	Implemented the decision-tree features, training, testing and validation for candidate suitability classification.
Web Implementation	Built the browser-based interface, APIs, candidate/job workflows, recommendation views and the mock-interview module.
Testing & Validation	Prepared test cases, benchmark experiments, performance measures and the comparative evaluation of matching approaches.
UI & Visualization	Designed the dashboard, A* search visualisation, recommendation explanations, skill-gap views and evaluation graphs.
Documentation & Deployment	Prepared figures, screenshots, technical documentation, reproducibility instructions and the final assignment report.

14. Individual Reflection

Working on SmartHire reshaped how I think about an AI recruitment system — from a simple keyword filter into a combination of search, learning and reasoning working together. The central design decision was to avoid handing every part of the recommendation to one algorithm. Structured matching handles transparent requirement checks, TF-IDF compares the raw text, A* satisfies the informed-search requirement, and the decision tree demonstrates learning from candidate-job features. Keeping these layers separate made it far easier to explain why a candidate ended up with a particular score.

Converting unstructured resumes into reliable features was the biggest implementation challenge. Resume layouts and terminology vary a lot, so the prototype leans on a controlled skill vocabulary and an alias list rather than assuming extraction is perfect. Making A* meaningful in a recruitment context was the second challenge — instead of forcing a physical shortest-path framing, I modelled job requirements as a state space where the search minimises unmet requirements, which gave the algorithm a clear heuristic and an observable, inspectable result.

The evaluation stage taught a useful lesson about metrics. The decision tree scored perfectly on the controlled synthetic split, but that is not evidence it would predict hiring outcomes perfectly in practice, since its labels were derived from the same structured scoring logic used to build the educational dataset. Likewise, the small ranking benchmark showed only a modest Precision@3 gain for the hybrid method — the stronger case for keeping the hybrid is that it supplies explicit requirement evidence and satisfies the assignment's informed-search objective.

15. References (APA 7th Edition)

1. Wang, J., Xu, Y., Liu, R., & Du, Y. (2026). Multi-task adversarial learning detects intersectional algorithmic bias in AI recruitment systems. Scientific Reports, 16, 22914. https://doi.org/10.1038/s41598-026-53457-9

2. Kumar, D., Verma, C., & Illés, Z. (2026). Optimizing student job placements with NLP and Explainable AI: A fair and transparent hiring framework. Array, 29, 100729. https://doi.org/10.1016/j.array.2026.100729

3. Spoladore, D., Criscuolo, S., & Isgrò, F. (2026). Representing jobs and job seekers in AI-based recommender systems: A literature review. Engineering Applications of Artificial Intelligence, 174, 114450. https://doi.org/10.1016/j.engappai.2026.114450

4. El-Deeb, R. H., Abdelmoez, W. M., & El-Bendary, N. (2026). Explainable and scalable job recommendation system using transformer-based text summarization, cross-encoder semantic similarity, and ensemble learning. Procedia Computer Science, 277, 1019–1030. https://doi.org/10.1016/j.procs.2026.02.142

5. Agrawal, P., Gandhi, S., Dattani, V., Rachchh, D., Rathod, M., & Vadher, K. (2026). Using machine learning to automate IT job role prediction by resume screening. In J. Choudrie, P. N. Mahalle, T. Perumal, & A. Joshi (Eds.), ICT for intelligent systems (Vol. 1517, pp. 71–84). Springer. https://doi.org/10.1007/978-981-96-8895-1_7

6. Zhu, C., Hu, X., Wu, H., Qin, C., Zhu, H., & Xiong, H. (2025). Enhancing job recommendations with LLM-based resume completion: A behavior-denoised alignment approach. Information Processing & Management, 62(6), 104261. https://doi.org/10.1016/j.ipm.2025.104261

7. Shiny, J. J., Sujitha, B., & Mithra, N. (2025). ResML: An improved resume screening and job recommendation system leveraging machine learning. In 2025 2nd Asia Pacific Conference on Innovation in Technology (APCIT) (pp. 1–6). IEEE. https://doi.org/10.1109/APCIT65661.2025.11411662

8. Singla, P., Kaur, J., Anju, Soni, A., Tuteja, A., & Sharma, S. (2024). Streamlining talent acquisition: A machine learning approach to automated resume screening. In 2024 2nd International Conference on Advanced Computing & Communication Technologies (ICACCTech) (pp. 1–6). IEEE. https://doi.org/10.1109/ICACCTech65084.2024.00022

9. Surya, C. H. S., Kiriti, G. L. V. S., Saketh, K. S., Charan, P. S., & Anjali, A. (2024). Optimizing job matching through automated resume screening with NLP and machine learning. In 2024 IEEE International Conference on Intelligent Signal Processing and Effective Communication Technologies (INSPECT) (pp. 877–882). IEEE. https://doi.org/10.1109/INSPECT63485.2024.10896014

10. Python Software Foundation. (n.d.). Python 3 documentation. https://docs.python.org/3/
11. Scikit-learn developers. (n.d.). Scikit-learn user guide. https://scikit-learn.org/stable/user_guide.html
12. Flask. (n.d.). Flask documentation. https://flask.palletsprojects.com/